Provenance in Preprocessing ML Pipelines

1

Before Talking about Provenance, let us take a look at the kinds of Modules (steps) that compose ML Pipelines

- Data Preprocessing Step: These implements operations that are similar to the relational algebra. In particular, we find the folloçwing kinds (see [Chapman et al., 2021] for more details):
  - Data transformation
  - Data reduction
  - Data augmention
- Predictor Step
  - Implement a model. Given an observation, data record, output a predition
- Generator Step
  - Given a set of (training) data records, it generates a predictor step (model).

2

Data Preprocessing Steps

Orange3 Ex.	ScikitLearn Ex.	Category	Operator	Implementation
Feature Statistics	Feature_selection	Data reduction	Feature Selection	π_C
Select Data by Index	Dataframe op.		Instance Selection	σ_C
Select Columns	Feature_selection		Drop Columns	π_C
Select Rows	Dataframe op.		Drop Rows	σ_C
Data Sampler	Imbalanced-learn		Undersampling	σ_C
Impute	SimpleImputer	Data transformation	Imputation	$\tau_{f(X)}$
Apply Domain	FunctionTransformer		Value Transformation	$\tau_{f(X)}$
Edit Domain	Binarizer		Binarization	$\tau_{f(X)}$
Preprocess	Normalizer		Normalization	$\tau_{f(X)}$
Discretize	KBinDiscretizer		Discretization	$\tau_{f(X)}$
Feature Constructor	FunctionTransformer	Data augmentation	Space Transformation	$\pi_Z \circ \alpha^{\rightarrow}_{f(X):Y}$
Create Class	FunctionTransformer		Instance Generation	$\alpha^{\downarrow}_{X:f(Y)}$
Data Sampler	Imbalanced-learn		Oversampling	$\alpha^{\downarrow}_{X:f(X)}$
Corpus	Label Encoder		String Indexer	$\alpha^{\rightarrow}_{f(X):Y}$
Preprocess	OneHotEncoder		One-Hot Encoder	$\alpha^{\rightarrow}_{f(X):Y}$

3

Data Reduction

- **Feature Selection**. This operation consists of selecting a set of relevant features from a given dataset and dropping the others, which are either redundant or irrelevant for the goal of the learning process.
- Feature selection over a dataset D with a schema S can be expressed by means of a simple pipeline involving only the projection operator with a condition that selects the set of features $I \subset S$ of interest:

$$FS(D) = \pi_C (D)$$

where $C = \{a \in I\}$.

- A special case of feature selection is an operation that drops columns with a value rate of missing values higher than a threshold t. In this case, the condition of the projection operator is more involved as it requires introspection of the dataset:

$$C = \{a \in S \mid count(D_{ia} = \perp, 1 \leq i \leq n) < t\}.$$

4

Data Reduction (cont.)

- **Instance Selection**. The aim of this operation is to reduce the original dataset to a manageable volume by removing noisy instances with the goal of improving the accuracy (and efficiency) of classification problems.
- Also in this case, instance selection over a dataset D with a schema S can be expressed by means of a simple pipeline involving only the selection operator with a condition that identifies the set of relevant rows of D by means of a predicate p:

  IS(D) = σC (D) where C = {Di$*$ ∈ S | p(Di$*$)}.

- Similar to feature selection, a relevant case of instance selection drops rows with a value rate of missing values higher than a threshold t. In this case,

  C={D_{i*} ∈ D | count(D_{ij} =⊥, 1≤j≤m) < t}

5

Data Transformation

Data transformation refers to any operation on a given dataset that modifies its values with the goal of improving the quality of D and/or making more effective the process of information extraction from D.

It can be expressed by means of a pipeline involving the data transformation operator:

DT(D) = τf(X)(D)

where$f$ can be any scalar function that associates with one or more values from the domain of the features X of S a value.

6

Data Transformation: Examples

- **Binarization**. It is the process of converting numerical features to binary features. For instance, if a value for a given feature is greater than a threshold it is changed a 1, if not to 0.
- **Normalization**. It is a scaling technique that transforms all the values of a feature so that they fall in a smaller range, such as from 0 to 1.
  - E.g., Min-Max normalization.
  - This operation operates on a single feature at a time
- **Discretization**. It performs a value transformation from categorical to numerical data.
- **Imputation**. It is the process of replacing missing data (nulls in our data model) with valid data using a variety of statistical approaches that aim at identifying the values with the maximum likelihood.

7

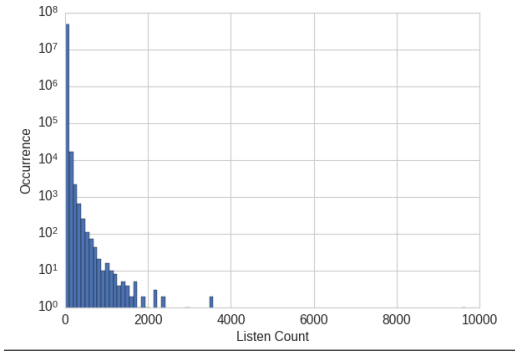
Dealing with counts

- Counts can quickly accumulate without bound.
- A user might put a song or a movie on infinite playback or use a script to repeatedly check for the availability of tickets for a popular show, which will cause the play count or website visit count to quickly rise.
- It is a good idea to check the scale and determine whether to keep the data as raw numbers, convert them into binary values to indicate presence, or bin them into coarser granularity.

8

Example

- The EchoNest [1] Dataset contains the full music listening histories of one million users on Echo Nest.
- Suppose our task is to build a recommender to recommend songs to users.
- One component of the recommender might predict how much a user will enjoy a particular song. Since the data contains actual listen counts, should that be the target of the prediction?
- However, the data shows that while 99% of the listen counts are 24 or lower, there are also some listen counts in the thousands, with the maximum being 9667. But more than 10,000 triplets have greater counts, with a few in the thousands.
- These values are anomalously large; if we were to try to predict the actual listen counts, the model would be pulled off course by these large values.

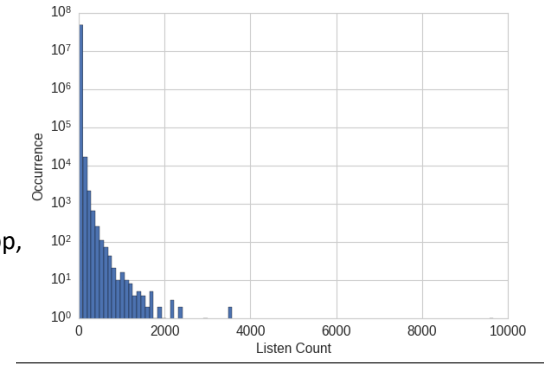


[1] http://millionsongdataset.com/tasteprofile/

9

Example

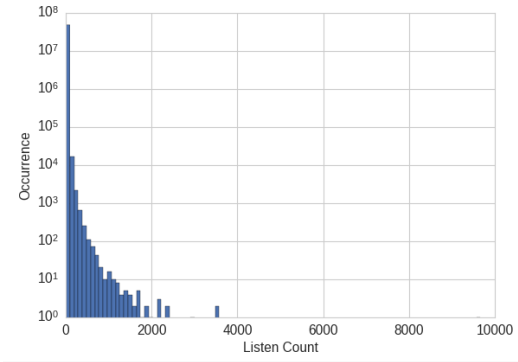
- The raw listen count is not a *robust* measure of user taste
  - Some people might put their favorite songs on infinite loop, while others might savor them only on special occasions.
  - We can't necessarily say that someone who listens to a song 20 times must like it twice as much as someone else who listens to it 10 times.
- A more robust representation of user preference is to binarize the count and clip all counts greater than 1 to 1.
- In other words, if the user listened to a song at least once, then we count it as the user liking the song.



[1] http://millionsongdataset.com/tasteprofile/

10

Example



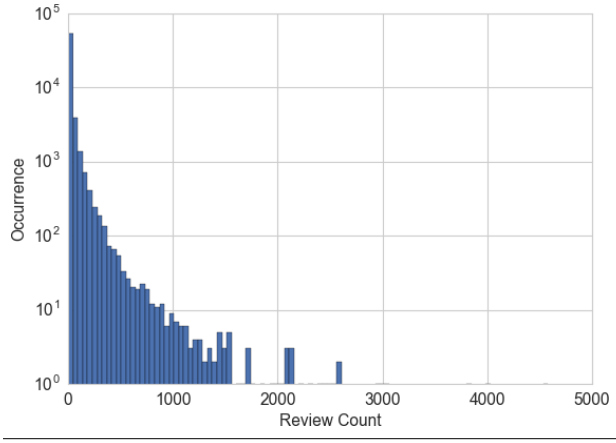

```
>>> import pandas as pd
>>> listen_count = pd.read_csv('millionsong/train_triplets.txt.zip',
...                            header=None, delimiter='\t')
# The table contains user-song-count triplets. Only nonzero counts are
# included. Hence, to binarize the count, we just need to set the entire
# count column to 1.
>>> listen_count[2] = 1
```

[1] http://millionsongdataset.com/tasteprofile/

11

Binning

- The Yelp dataset [2] contains user reviews of businesses from 10 cities across North America and Europe.
  - Each business has a review count.
- Suppose our task is to predict the rating a user might give to a business.
- The review count might be a useful input feature because there is usually a strong correlation between popularity and good ratings.
- For this purpose, should we use the raw review count or process it further?



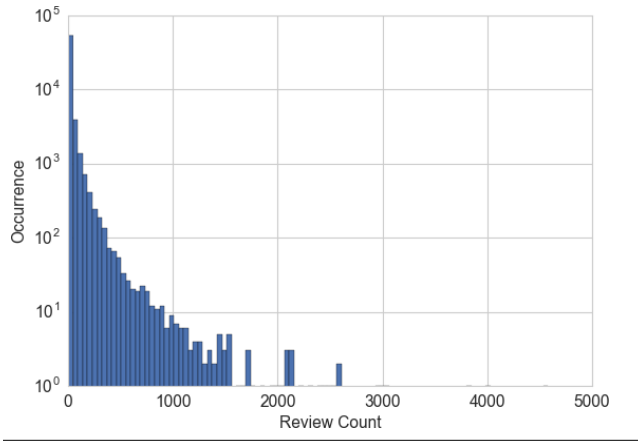
The answer is no.
As in the listen counts, most of the counts are small, but some businesses have reviews in the thousands.

[2] https://www.yelp.com/dataset

12

Binning

- One solution is to contain the scale by binning the count.
- In other words, we group the counts into bins, and get rid of the actual count values.
- Binning maps a continuous number to a discrete one.
- We can think of the discretized numbers as an ordered sequence of bins that represent a measure of intensity.
- To bin data, we have to decide how wide each bin should be.
  - fixed-width or adaptive.



[2] https://www.yelp.com/dataset

13

Fixed-width binning

- With fixed-width binning, each bin contains a specific numeric range. The ranges can be custom designed or automatically segmented, and they can be linearly scaled or exponentially scaled.

Linearly scaled bins

- 0–12 years old
- 12–17 years old
- 18–24 years old
- 25–34 years old
- 35–44 years old
- 45–54 years old
- 55–64 years old

Exponentially scaled bins

- 0–9, 10^1
- 10–99, 10^2
- 100–999, 10^3
- 1000–9999, 10^4
- etc

14

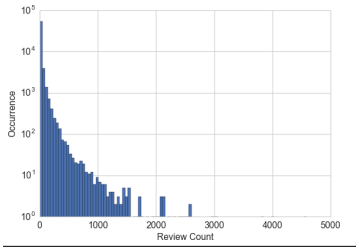
Quantile binning

- If there are large gaps in the counts, then there will be many empty bins with no data.
- This problem can be solved by adaptively positioning the bins based on the distribution of the data.
- This can be done using the quantiles of the distribution.
- Quantiles are values that divide the data into equal portions.
  - The median divides the data in halves; half the data points are smaller and half larger than the median.
  - The quartiles divide the data into quarters,
  - the deciles into tenths, etc.

15

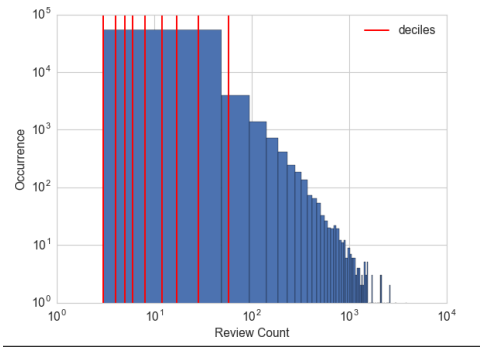
Quantile binning (Example)

- Compute the deciles of the Yelp business review counts,




```
>>> deciles = biz_df['review_count'].quantile([.1, .2, .3, .4, .5, .6, .7, .8, .9])
>>> deciles
0.1     3.0
0.2     4.0
0.3     5.0
0.4     6.0
0.5     8.0
0.6    12.0
0.7    17.0
0.8    28.0
0.9    58.0
```

The figure on the right overlays the deciles on the histogram. This gives a much clearer picture of the skew toward smaller counts.



16

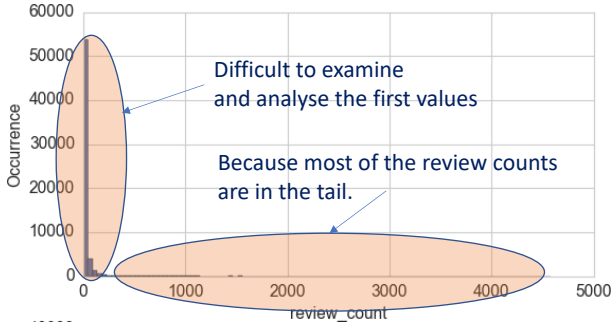
Log Transformation

- The log function is the inverse of the exponential function: $\log_a(a^x) = x$,
- Since $a^0 = 1$, we have $\log_a(1) = 0$.
- This means that the log function maps the small range of numbers between (0, 1) to the entire range of negative numbers (–∞, 0).
- The function $\log_{10}(x)$ maps the range
  - [1, 10] to [0, 1],
  - [10, 100] to [1, 2],
  - and so on.
- In other words, the log function compresses the range of large numbers and expands the range of small numbers. The larger x is, the slower log(x) increments.

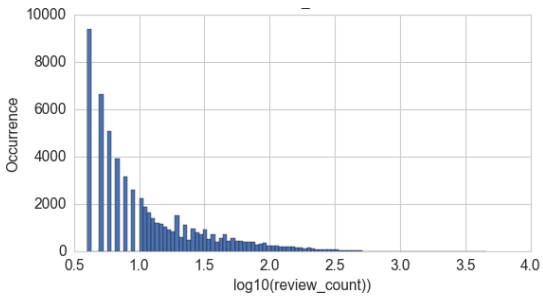
17

Log Transformation

- But why are we talking about the log function?
- Log transformation helps reducing skewness when you have skewed data.
- In particular, the log transform is a powerful tool for dealing with positive numbers with a heavy- tailed distribution.
- This is the case for the review counts.
- Take a look at the distribution in the figure on the left



The log transformation provides a better idea on the distribution



18

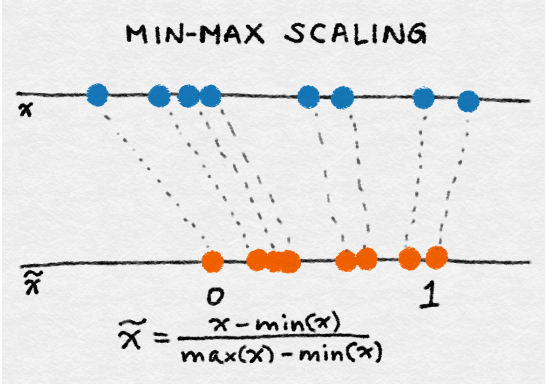
Feature Scaling or Normalization

- Feature scaling (also called feature normalization) changes the scale of the feature.

Min-Max Scaling

- Let *x* be an individual feature value, and min(*x*) and max(*x*), respectively, be the minimum and maximum values of this feature over the entire dataset. M
- Min-max scaling squeezes (or stretches) all feature values to be within the range of [0, 1]. The formula for min-max scaling is:

$$\tilde{x} = \frac{x - \min(x)}{\max(x) - \min(x)}$$



19

Data Augmentation

- **Space Transformation**. This operation takes a set of features of an existing dataset and generates from them a new set of features by combining the corresponding values. Usually, the goal is to represent (a subset of) the original set of features in terms of others in order to increase the quality of learning.
- The application of this operation to a dataset D over a schema S can be expressed by means of a expression involving a vertical augmentation that operates on a subset X of the features in S and produce a new set of features Y, followed by a projection operator that eliminates the features in X , thus maintaining those in $Z = (S \cup Y) - X$:

$$ST(D) = \pi_{\{features\ in\ Z\}}(\alpha^{\rightarrow}_{f(X):Y}(D))$$

20

Data Augmentation (cont.)

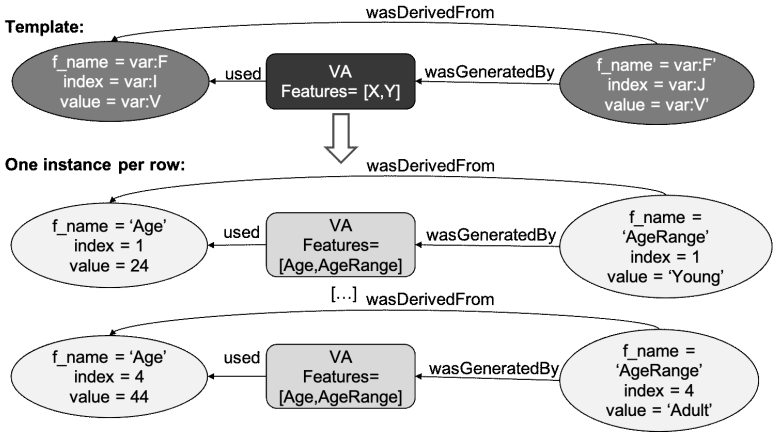
- **Instance Generation**: This process allows us to fill regions in the domain of the problem, which do not have representative examples in original data, or to summarize large amounts of instances in fewer examples.
- The application of this operation to a dataset D over a schema S can be expressed by means of a expression involving a horizontal augmentation S:

$$IG(D) = \alpha^{\downarrow}_{X:f(Y)}(D).$$

21

Provenance Capture: Templates

Example for vertical augmentation (VA)



22

For more information on provenance capture for data pre-processing steps

- Github repository:

https://github.com/GiuliaSim/DataProvenance

24

It seems that Data Processing can be traced using provenance, how about the other steps?

- Data Preprocessing Step: These implements operations that are similar to the relational algebra. In particular, we find the folloçwing kinds (see [Chapman et al., 2021] for more details):

- Predictor Step -> Existing provenance capture model are coarse-grained
  - Impelement a model. Given an observation, data record, output a predition
- Generator Step -> Existing provenance capture model are coarse-grained
  - Given a set of (training) data records, it generates a predictor step (model).

25

Provenance Size is an Issue

	Input size (kb)	Provenance size	Provenancesize/Inputsize
Census	3700	3100000	837.84
Compas	2500	161000	64.40
German	79	59000	746.84

26

mlinspect

Objective: help diagnose and mitigate **technical bias** that arises during **preprocessing steps in an ML** pipeline.

1. Preexisting bias: under- or over-representation of particular groups in the training data.
2. Technical bias: E.g., skew introduced during data preparation

The focus in mlinspect is on (2).

- Preprocessing can introduce skew in the data, and, in particular, it can exacerbate under-representation of historically disadvantaged groups.
  - filters or joins can heavily change the distribution of different groups represented in the training data.
  - missing value imputation can also introduce skew.
- ML fairness research, which mostly focuses on static datasets, is not sufficient because it cannot address such technical bias originating from the data preparation stage.

27

Mlinspect: example

- Consider a data scientist who implements a Python pipeline that takes demographic and clinical history data as input, and trains a classifier to identify patients at risk for serious complications.
  - a legal obligation to ensure that the resulting model works equally well for patients across different age groups and races.
  - This is operationalized as an intersectional fairness criterion, requiring equal **false-negative rates** for groups of patients identified by a combination of age_group and race.
- Steps of the pipeline
  1. reads two CSV files, demographics and clinical histories of patient, respectively. The resulting dataframes are joined on the ssn column.

     => This join may introduce a data distribution bug: some combination of age group and race do not have match- ing entries in the clinical history dataset
  2. computes the average number of complications per age group and adds the binary target label to the dataset, indicating which patients had a higher than average number of complications compared to their age group.
  3. Data is then projected to a subset of the attributes, to be used by the classification model.

     ⇒The age group is projected out, however, it is needed to check fairness!
  4. filters the data to only contain records from patients within a given set of counties.

     ⇒ a data distribution bug may be introduced if populations of different counties systematically differ in age.

.....

28

mlinspect: Example

Provenance capture is a pre-requisite for the checks

Potential issues in preprocessing pipeline:

1. Join might change proportions of groups in data
2. Column 'age_group' projected out, but required for fairness
3. Selection might change proportions of groups in data
4. Imputation might change proportions of groups in data
5. 'race' as a feature might be illegal!
6. Embedding vectors may not be available for rare names!

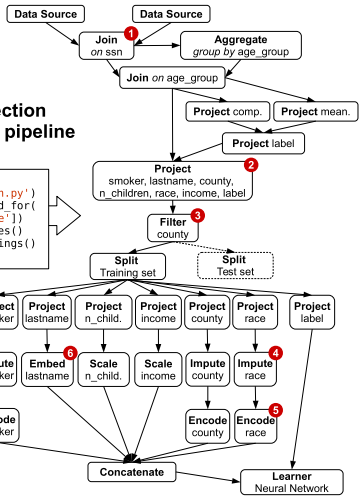
Python script for preprocessing, written exclusively with native pandas and sklearn constructs

```
# load input data sources, join to single table
patients = pandas.read_csv(…)
histories = pandas.read_csv(…)
data = pandas.merge([patients, histories], on=['ssn'])

# compute mean complications per age group, append as column
complications = data.groupby('age_group')
  .agg(mean_complications=('complications','mean'))
data = data.merge(complications, on=['age_group'])

# Target variable: people with frequent complications
data['label'] = data['complications'] >
  1.2 * data['mean_complications']

# Project data to subset of attributes, filter by counties
data = data[['smoker', 'last_name', 'county',
             'num_children', 'race', 'income', 'label']]
data = data[data['county'].isin(counties_of_interest)]

# Define a nested feature encoding pipeline for the data
impute_and_encode = sklearn.Pipeline([
  (sklearn.SimpleImputer(strategy='most_frequent')),
  (sklearn.OneHotEncoder())])
featurisation = sklearn.ColumnTransformer(transformers=[
(impute_and_encode, ['smoker', 'county', 'race']),
(Word2VecTransformer(), 'last_name')
(sklearn.StandardScaler(), ['num_children', 'income']])

# Define the training pipeline for the model
neural_net = sklearn.KerasClassifier(build_fn=create_model())
pipeline = sklearn.Pipeline([
  ('features', featurisation),
  ('learning_algorithm', neural_net)])

# Train-test split, model training and evaluation
train_data, test_data = train_test_split(data)
model = pipeline.fit(train_data, train_data.label)
print(model.score(test_data, test_data.label))
```

Declarative inspection of preprocessing pipeline

```
mlinspect
PipelineInspector
.on_pipeline('health.py')
.no_bias_introduced_for(
['age_group', 'race'])
.no_illegal_features()
.no_missing_embeddings()
.verify()
```

Corresponding dataflow DAG for instrumentation, extracted by *mlinspect*



29

BugDoc: Algorithms to Debug Computational Processes

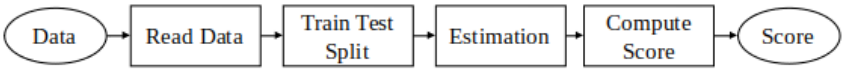
Problem definition: Given a pipeline that fails (the failure can be a property that is explicitely defined, e.g., the accuracy of a prediction model is below 6, or some steps in the pipeline do not terminates normally), identify the parameter-values that source of the problem.

- This work is relevant for machine learning pipelines where the parameter (or hyper-parameter) may lead to a failed execution of the pipelines, and the objective is to help the scientist debug the root of the failure.
  - **Use case 1**: Enterprise Analytics. Plots for sales forecasts showed a sharp decrease compared to historical values. The problem was tracked down to a data feed, whose temporal resolution had changed from monthly to weekly. This in turn affected the predictions of ML pipelines, leading to incorrect forecasts.
  - **Use case 2**: Exploring Supernovas. In an astronomy experiment, some visualizations of supernovas presented unusual arti- facts that could have indicated a discovery. The problem was due to a new version of a data processing software.

To debug such problems, users currently expend considerable effort reasoning about the effects of the many possible different settings. This requires them to tune and execute new pipeline instances to test hypotheses manually, which is tedious ande time-consuming.

30

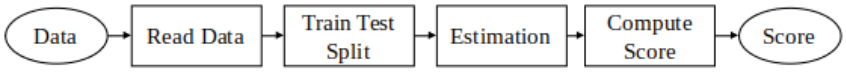
30



Instance	Data	Estimator	Library Version	Score	Evaluation
CP_1	Iris	Logistic regression	1.0	0.9	Success
CP_2	Digits	Decision tree	1.0	0.8	Success
CP_3	Iris	Gradient boosting	2.0	0.2	Failure
CP_4	Digits	Gradient boosting	2.0	0.3	Failure
CP_5	Iris	Decision tree	1.0	0.7	Success
CP_6	Images	Gradient boosting	1.0	0.9	Success

- Gradient boosting leads to low scores for two of the datasets (Iris and Digits), but it has a high score for Images.
- By contrast, decision trees work well for both the Iris and Digits datasets, and logistic regression leads to a high score for Iris.

31



Instance	Data	Estimator	Library Version	Score	Evaluation
CP_1	Iris	Logistic regression	1.0	0.9	Success
CP_2	Digits	Decision tree	1.0	0.8	Success
CP_3	Iris	Gradient boosting	2.0	0.2	Failure
CP_4	Digits	Gradient boosting	2.0	0.3	Failure
CP_5	Iris	Decision tree	1.0	0.7	Success
CP_6	Images	Gradient boosting	1.0	0.9	Success

- This may suggest that:
  - there is a problem with the gradient boosting module for some parameters
- A definitive conclusion has to await additional testing of these hyperparameters.
- Doing so manually is time- consuming and error-prone
- BugDoc aims to automate this process.

32

BugDoc

- A naïve approach to discovering the root of the problem would be to to try every possible parameter-value pair combination of the parameter-value universe
  $\Rightarrow$testing of a number of pipeline instances that is exponential in the number of parameters.
- BugDoc explores a subset of the space:
  1. It starts from a pipeline instance CP_f that evaluates to fail.
  2. It then uses pipeline instances that succeeded and are disjoint , i.e., they share no parameter-values, from CP_f to construct new tests. Let CP_g be such instance.
  3. The current instance $CP_{current}$ is initialized to CP_f .
  4. Using some order among parameters, for each parameter p, an instance $CP'_{current}$ is created such that $CP'_{current}[p]=CP_g[p]$
  5. If the instance $CP'_{current}$ fails then $CP_{current}$ is changed to $CP'_{current}$ and the next parameter is considered.
     - The intution is that the value of p in CP_f did not cause the failure.
  6. The root cause will be a subset of the pipeline instance CP_f that is still present in the final instance of $CP_{current}$

33

BugDoc: Example

Instance	Data	Estimator	Library Version	Score	Evaluation
CP_1	Iris	Logistic regression	1.0	0.9	Success
CP_2	Digits	Decision tree	1.0	0.8	Success
CP_3	Iris	Gradient boosting	2.0	0.2	Failure
CP_4	Digits	Gradient boosting	2.0	0.3	Failure
CP_5	Iris	Decision tree	1.0	0.7	Success
CP_6	Images	Gradient boosting	1.0	0.9	Success

In the initial traces shown in the above Table, there are only two disjoint instances with different evaluations:

CP_g = {(Dataset,Digits), (Estimator,Decision Tree), (LibraryVersion,1.0)}

CP_f ={(Dataset,Iris), (Estimator,Gradient Boosting), (LibraryVersion,2.0) }

Dataset	Estimator	Library Version	Score	Evaluation ($score \geq 0.6$)
Iris	Logistic Regression	1.0	0.9	succeed
Digits	Decision Tree	1.0	0.8	succeed
Iris	Gradient Boosting	2.0	0.2	fail
Digits	Gradient Boosting	2.0	0.2	fail
Digits	Decision Tree	2.0	0.3	fail
Digits	*Decision Tree*	*1.0*	*0.8*	succeed

As well as the above algorithm, BugDog proposes another algorithm based on decision trees

34

The three proposals we have just seen have different focuses

- Chapman et al.: The focus here is on capturing fine grained provenance using the standards prov, and on allowing users to query recorded provenance.
- Mlinspect: Aims to check some properties (distributions) in particular with fairness in mind.
- BigDoc: inspect the root of errors that a pipeline may suffer from by exploring different settings (pipeline instances).

There are also other proposals, and this seems to be a hot topic of research….

35

- MLINSPECT
  - Stefan Grafberger, Shubha Guha, Julia Stoyanovich, Sebastian Schelter: MLINSPECT: A Data Distribution Debugger for Machine Learning Pipelines. SIGMOD Conference 2021: 2736-2739
  - Stefan Grafberger, Julia Stoyanovich, Sebastian Schelter: Lightweight Inspection of Data Preprocessing in Native Machine Learning Pipelines. CIDR 2021
  - Stefan Grafberger, Paul Groth, Julia Stoyanovich, Sebastian Schelter: Data distribution debugging in machine learning pipelines. VLDB J. 31(5): 1103-1126 (2022)
  - Sebastian Schelter, Stefan Grafberger, Shubha Guha, Olivier Sprangers, Bojan Karlas, Ce Zhang: Screening Native Machine Learning Pipelines with ArgusEyes. CIDR 2022
- Dagger
  - El Kindi Rezig, Lei Cao, Giovanni Simonini, Maxime Schoemans, Samuel Madden, Nan Tang, Mourad Ouzzani, Michael Stonebraker: Dagger: A Data (not code) Debugger. CIDR 2020
  - El Kindi Rezig, Ashrita Brahmaroutu, Nesime Tatbul, Mourad Ouzzani, Nan Tang, Timothy G. Mattson, Samuel Madden, Michael Stonebraker: Debugging Large-Scale Data Science Pipelines using Dagger. Proc. VLDB Endow. 13(12): 2993-2996 (2020)
- BugDoc
  - Raoni Lourenço, Juliana Freire, Dennis E. Shasha: BugDoc: Algorithms to Debug Computational Processes. SIGMOD Conference 2020: 463-478
  - Raoni Lourenço, Juliana Freire, Dennis E. Shasha: Debugging Machine Learning Pipelines. DEEM@SIGMOD 2019: 3:1-3:10

36

Capturing and Using Provenance in Data Preprocessing ML Pipelines

- Capturing Provenance (Prov)
  - Adriane Chapman, Paolo Missier, Giulia Simonelli, Riccardo Torlone: Capturing and querying fine-grained provenance of preprocessing pipelines in data science. Proc. VLDB Endow. 14(4): 507-520 (2020)
  - Adriane Chapman, Paolo Missier, Giulia Simonelli, Riccardo Torlone: Fine-grained Provenance for High-quality Data Science (Discussion Paper). SEBD 2021: 411-418
- Juneau
  - Yi Zhang, Zachary G. Ives: Juneau: Data Lake Management for Jupyter. Proc. VLDB Endow. 12(12): 1902-1905 (2019)
  - Yi Zhang, Zachary G. Ives: Finding Related Tables in Data Lakes for Interactive Data Science. SIGMOD Conference 2020: 1951-1966
- ProVision
  - Nan Zheng, Abdussalam Alawini, Zachary G. Ives: Fine-Grained Provenance for Matching & ETL. ICDE 2019: 184-195
- DiagnosingML
  - Zhao Zhang, Evan R. Sparks, Michael J. Franklin: Diagnosing Machine Learning Pipelines with Fine-grained Lineage. HPDC 2017: 143-153

37

Github repositories

- Chapman et al.
  https://github.com/GiuliaSim/DataProvenance.
- Mlinspect
  https://github.com/stefan-grafberger/mlinspect
- BugDoc
  https://github.com/VIDA-NYU/BugDoc

38